Hacking ético con Docker

Manual de estudio progresivo

Del laboratorio aislado a la comprobación de controles de acceso

Edición 1.0 · 7 de noviembre de 2026 · Idioma: español

Un recorrido práctico por Docker, redes, HTTP, terminal, expresiones regulares, JavaScript, APIs, JWT y documentación de hallazgos.

Entorno de práctica: OWASP Juice Shop en un laboratorio propio. Los ejemplos de reconocimiento apuntan exclusivamente al servicio local `juice-shop:3000`.

Cómo estudiar: lee la explicación, anticipa el resultado del comando, ejecútalo en tu laboratorio y compara la respuesta. Conserva tanto las observaciones como las dudas: una hipótesis no es una vulnerabilidad demostrada.

Documento vivo: la fuente Markdown que acompaña a este PDF permite añadir módulos, corregir ejemplos y registrar nuevas evidencias sin perder la estructura del curso.

OBSERVAR · COMPRENDER · REPRODUCIR · DOCUMENTAR

01 · Alcance, evidencia y ruta de aprendizaje

Qué procede de la conversación

Se recuperaron los últimos intercambios del curso. En ellos consta un `200` al consultar `/api/Users` con `CUSTOMER_TOKEN`, una respuesta completa de `/api/Users/1` y la confirmación del alumno de que la iteración devolvió cinco usuarios.

El historial disponible no contiene la instalación inicial ni todos los comandos anteriores. Los capítulos sobre esos temas son una **reconstrucción pedagógica basada en el temario solicitado**, no una transcripción ni una afirmación de que esa configuración exacta se ejecutó. No se han probado aquí los comandos contra tu VPS.

Usaremos tres etiquetas: **Evidencia observada** para la respuesta conservada; **Resultado comunicado** para lo confirmado por el alumno; **Ejemplo orientativo** para salidas que dependen de la versión y del estado del laboratorio.

Itinerario del manual

1. Objetivos, reglas y preparación segura del VPS (págs. 3-4).
2. Carpetas, Dockerfile, Compose y redes (págs. 5-8).
3. Reconocimiento con Nmap y lectura de HTTP con curl (págs. 9-11).
4. Pipes, grep, expresiones regulares y recursos JavaScript (págs. 12-15).
5. Endpoints, JSON y filtros jq (pág. 16).
6. Login, tokens JWT y almacenamiento en el navegador (págs. 17-19).
7. Autenticación, autorización y acceso por identificador (págs. 20-22).
8. Informe, mitigación, próximos módulos y fuentes (págs. 23-26).

Convenciones

Los comandos indican su contexto: **VPS**, **equipo local** o **contenedor de herramientas**. No copies el prompt `root@...:/lab#` ni el signo `>` que Bash muestra al continuar una línea. Las barras `\` al final unen varias líneas: no deben tener espacios después.

Las rutas, versiones y credenciales de ejemplo se deben adaptar. Nunca sustituyas el destino del laboratorio por un sistema ajeno sin autorización expresa.

02 · Objetivos y reglas del laboratorio

Qué aprenderás

Comprenderás cómo se comunican los contenedores, qué revela un servicio HTTP y cómo pasar de una pista en JavaScript a una petición reproducible. Aprenderás a distinguir identidad, rol y permisos sobre cada objeto, y a describir el alcance real de un hallazgo.

El objetivo práctico alcanzado en la conversación es observar que un usuario de tipo `customer` pudo leer registros de otros usuarios mediante identificadores de la API.

Reglas de trabajo

- Trabaja solo con tu laboratorio o con un alcance autorizado por escrito.
- Usa cuentas y contraseñas exclusivas de prácticas; no introduzcas datos personales reales.
- Mantén Juice Shop fuera de Internet: es una aplicación deliberadamente vulnerable.
- Limita las peticiones a lo necesario. El bucle de cinco IDs demuestra el comportamiento sin recorrer toda la base.
- Guarda evidencias mínimas y oculta tokens y contraseñas en capturas e informes.
- No montes el socket Docker ni carpetas sensibles del VPS dentro del laboratorio.
- Detén los servicios cuando acabes y conserva las notas fuera de contenedores desechables.

Modelo de amenazas básico

El VPS es una máquina expuesta a Internet. Los contenedores comparten su núcleo: no equivalen a máquinas virtuales con aislamiento absoluto. Una aplicación vulnerable puede poner en riesgo el host si dispone de privilegios, montajes o conectividad innecesarios.

Nota de seguridad: utiliza un VPS dedicado a prácticas, actualizado y sin servicios de producción. Evita `privileged: true`, `network_mode: host` y publicar el daemon Docker por TCP. El grupo `docker` permite operaciones equivalentes a administración del host; no es un permiso inocuo.

Pregunta de control

¿Una red Docker privada basta para proteger una aplicación si se publica `3000:3000`? No. Esa publicación puede exponer el puerto en todas las interfaces del VPS. La siguiente parte separa aislamiento de red y publicación de puertos.

03 · Instalación de Docker en el VPS

Preparación y verificación

Contexto: VPS. Este ejemplo presupone Ubuntu compatible con Docker Engine. Si usas Debian u otra distribución, sigue la guía correspondiente; no mezcles repositorios. Comprueba primero el sistema y si Docker ya está instalado.

```
cat /etc/os-release
docker --version
docker compose version
```

Para una instalación nueva, consulta la guía oficial [D1], resuelve los paquetes incompatibles que indique y configura su repositorio firmado. Secuencia orientativa para Ubuntu:

```
sudo apt update
sudo apt install -y ca-certificates curl
sudo install -m 0755 -d /etc/apt/keyrings
sudo curl -fsSL https://download.docker.com/linux/ubuntu/gpg \
  -o /etc/apt/keyrings/docker.asc
sudo chmod a+r /etc/apt/keyrings/docker.asc
. /etc/os-release
sudo tee /etc/apt/sources.list.d/docker.sources >/dev/null <<EOF_DOCKER
Types: deb
URIs: https://download.docker.com/linux/ubuntu
Suites: ${UBUNTU_CODENAME:-$VERSION_CODENAME}
Components: stable
Architectures: $(dpkg --print-architecture)
Signed-By: /etc/apt/keyrings/docker.asc
EOF_DOCKER
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io \
  docker-buildx-plugin docker-compose-plugin
sudo docker run --rm hello-world
sudo docker compose version
```

Resultado orientativo: `hello-world` confirma que el motor descarga y ejecuta una imagen; Compose muestra su versión. Esto no valida todavía el aislamiento del laboratorio.

Protege el acceso SSH con claves y restringe su origen en el cortafuegos del proveedor. Prueba una segunda sesión antes de cerrar la primera si cambias SSH o el firewall. No abras el puerto 3000 públicamente. Docker puede eludir reglas habituales de UFW al publicar puertos [D1]; verifica la publicación efectiva.

04 · Carpetas y significado de mkdir -p

Contexto: VPS. Crea un directorio específico para el curso:

```
mkdir -p ~/ethical-lab/tools ~/ethical-lab/work
cd ~/ethical-lab
```

`mkdir` crea directorios. `-p` crea también los padres que falten y no falla porque el directorio ya exista. No borra su contenido y sí puede fallar por permisos o porque un componente sea un archivo. `~` representa el directorio personal del usuario actual.

Estructura propuesta

```
ethical-lab/
  compose.yaml          # servicios y red
  tools/
    Dockerfile          # imagen de herramientas
  work/                 # capturas y archivos de práctica
```

Esta estructura es una propuesta reproducible, no la estructura exacta recuperada del historial. El contenedor de herramientas verá `work/` como `/lab`; los archivos que escribas allí permanecerán en el VPS.

Tres conceptos diferentes

Imagen: plantilla con sistema de archivos y configuración. **Contenedor:** instancia ejecutable de una imagen. **Volumen o bind mount:** almacenamiento que puede sobrevivir a la eliminación de esa instancia.

El Dockerfile construye una imagen. Compose describe cómo iniciar y conectar servicios. El directorio de trabajo es donde guardas los resultados; no forma parte automáticamente de una imagen.

Nota práctica: ejecutar las herramientas como root dentro del contenedor puede crear archivos propiedad de root en `work/`. Esa identidad no convierte al contenedor en una máquina independiente ni justifica darle privilegios adicionales.

Ejercicio

Ejecuta `mkdir -p` dos veces y después `ls -la ~/ethical-lab`. Debes encontrar los directorios una sola vez y conservar los archivos que ya existieran.

05 · Dockerfile: construir las herramientas

Guarda este contenido en `tools/Dockerfile` desde el VPS:

```
FROM debian:bookworm-slim
RUN apt-get update \
 && apt-get install -y --no-install-recommends \
      bash ca-certificates curl nmap jq python3 grep \
 && rm -rf /var/lib/apt/lists/*
WORKDIR /lab
CMD ["sleep", "infinity"]
```

Lectura línea por línea

`FROM` elige la imagen base. La variante `slim` reduce el contenido inicial. Una etiqueta puede cambiar con el tiempo; registra el digest si necesitas repetir exactamente el entorno.

`RUN` ejecuta instrucciones durante la construcción. `&&` continúa solo si la instrucción anterior tuvo éxito. La actualización del índice y la instalación comparten una capa para evitar reutilizar un índice antiguo por error.

`--no-install-recommends` evita paquetes recomendados innecesarios. La limpieza final elimina índices de paquetes, no las herramientas instaladas.

`WORKDIR /lab` establece el directorio de trabajo. `CMD` define el proceso por defecto: `sleep infinity` mantiene el contenedor disponible para abrir una sesión con Compose. No inicia ninguna auditoría.

Construcción frente a ejecución

Las descargas de `apt` ocurren al construir la imagen. Después, el laboratorio puede ejecutarse en una red con `internal: true`. No necesitas permitir salida a Internet a la aplicación vulnerable para instalar sus herramientas en cada sesión.

Resultado esperado: la construcción termina sin errores y la imagen contiene `curl`, `nmap`, `jq` y Python. Si falla, revisa conectividad del host, disponibilidad del repositorio y espacio en disco. No suprimas verificaciones de certificados para resolverlo.

06 · Docker Compose y red del laboratorio

Guarda este ejemplo como `compose.yaml` en `~/ethical-lab`:

```
services:
  juice-shop:
    image: bkimminich/juice-shop:latest
    ports:
      - "127.0.0.1:3000:3000"
    networks:
      - lab
    security_opt:
      - no-new-privileges:true

  tools:
    build: ./tools
    volumes:
      - ./work:/lab
    networks:
      - lab
    cap_drop:
      - ALL
    security_opt:
      - no-new-privileges:true

networks:
  lab:
    driver: bridge
    internal: true
```

`services` agrupa los contenedores. `image` descarga una imagen ya construida; `build` construye la de herramientas. `volumes` enlaza una carpeta del VPS con `/lab`. Ambos servicios comparten `lab` y pueden resolver el nombre `juice-shop` mediante el DNS de Docker [D3].

Una red bridge separada limita qué contenedores se comunican directamente. `internal: true` añade aislamiento de conectividad externa; no cifra tráfico ni sustituye el control de acceso de la aplicación. El host sigue siendo una frontera de confianza.

`latest` facilita la primera descarga, pero no fija una versión. Para conservar los resultados del curso, registra la imagen y reemplaza la referencia por un tag probado o su digest. Los endpoints y vulnerabilidades pueden variar al actualizar.

Nota: esta configuración reduce privilegios del contenedor de herramientas. Por ello emplearemos Nmap con conexión TCP `-sT`, sin depender de paquetes raw.

07 · Puertos, acceso y ciclo de vida

Contexto: VPS, desde `~/ethical-lab`:

```
sudo docker compose config
sudo docker compose up -d --build
sudo docker compose ps
sudo docker compose logs --tail=40 juice-shop
sudo docker compose exec tools bash
```

`config` valida y resuelve la configuración. `up -d --build` construye lo necesario e inicia los servicios en segundo plano. `ps` muestra su estado y puertos; `logs` ayuda a comprobar el arranque. `exec` abre Bash en un contenedor ya iniciado. Escribe `exit` para volver al VPS.

Qué significa 127.0.0.1:3000:3000

El primer valor es la interfaz del **VPS**, el segundo su puerto y el tercero el puerto del contenedor. Desde el VPS se usa `http://127.0.0.1:3000`; desde `tools`, `http://juice-shop:3000`. Dentro de `tools`, `127.0.0.1` apunta a `tools`, no a Juice Shop.

Contexto: equipo local. Abre un túnel SSH y mantenlo activo:

```
ssh -N -L 3000:127.0.0.1:3000 usuario@IP_DEL_VPS
```

Visita `http://127.0.0.1:3000` en tu navegador local. Si el puerto local está ocupado, cambia solo el primer `3000` por `3001` y abre ese puerto en el navegador. La conexión entre tu equipo y el VPS viaja cifrada por SSH.

Verificación y cierre

Comprueba que `compose ps` publica en `127.0.0.1`, sin `0.0.0.0` ni `[::]`. Usa Docker actualizado; la documentación registra limitaciones de localhost en versiones anteriores a 28.0.0 [D2]. La prueba desde un equipo externo autorizado no debe poder alcanzar `IP_DEL_VPS:3000`.

En el VPS, `sudo docker compose stop` detiene; `start` reanuda. `down` elimina contenedores y la red del proyecto. Los datos internos de Juice Shop pueden perderse al recrearlo; el bind mount `work/` permanece. Evita añadir `-v` sin comprender qué volúmenes borraría.

08 · Reconocimiento con Nmap

Contexto: contenedor de herramientas. Limita la prueba al servicio del laboratorio:

```
nmap --unprivileged -sT -sV -p 3000 juice-shop
```

`--unprivileged` evita que Nmap presuponga privilegios raw por ejecutarse como root. `-sT` usa conexiones TCP normales. `-p 3000` limita el puerto. `-sV` envía sondas para intentar identificar el servicio y su versión; no significa «buscar vulnerabilidades» [N1]. La identificación es una inferencia basada en respuestas.

Salida orientativa

```
Nmap scan report for juice-shop (172.20.0.2)
Host is up (0.00010s latency).
PORT     STATE SERVICE VERSION
3000/tcp open  http    [identificación variable]
```

La IP es ilustrativa. `Host is up` indica que el destino respondió al mecanismo usado, no que esté sano. `3000/tcp` identifica puerto y protocolo. `open` significa que acepta conexiones; `closed`, que responde pero no hay servicio escuchando; `filtered`, que Nmap no puede resolver el estado porque las sondas se filtran o no obtiene respuesta suficiente.

`SERVICE` y `VERSION` pueden ser imprecisos, estar vacíos o incluir interrogantes. Sin detección activa, un nombre de servicio puede ser solo la asociación habitual de un número de puerto. No atribuyas una versión exacta si la salida no la establece.

Del puerto a HTTP

Confirma la hipótesis con `curl -i http://juice-shop:3000/`. Un servicio HTTP suele devolver una línea de estado, cabeceras y un cuerpo. Guarda la salida de Nmap con fecha y contexto si forma parte de un informe.

Diagnóstico: si el nombre no se resuelve, revisa que ambos servicios estén en la misma red. Si la conexión se rechaza, revisa el arranque y el puerto. No amplíes automáticamente el escaneo a la red del proveedor.

09 · HTTP con curl: leer la respuesta

Contexto: herramientas. Define una base reutilizable:

```
BASE='http://juice-shop:3000'
curl -I "$BASE/"
curl -i "$BASE/"
curl -sS "$BASE/"
```

`-I` solicita **HEAD**: pide cabeceras sin cuerpo. No equivale a hacer GET y ocultar su cuerpo; algunos servidores tratan HEAD de forma diferente. `-i` incluye las cabeceras junto al cuerpo de la petición normal. `-s` silencia el progreso y los mensajes de error; `-sS` conserva los errores y es mejor para aprender y automatizar [C1].

Anatomía de una respuesta

```
HTTP/1.1 200 OK
Content-Type: application/json; charset=utf-8
Content-Length: 123

{"status":"success","data":{}}
```

Es un ejemplo sintético. La primera línea contiene protocolo y estado; después vienen las cabeceras; una línea vacía separa el cuerpo. `Content-Type` describe el formato. `Content-Length`, cuando existe, indica bytes, no caracteres.

`200` indica éxito HTTP; `401`, ausencia de autenticación aceptada; `403`, acceso rechazado; `404`, recurso no encontrado o deliberadamente oculto. Un `500` sugiere error del servidor. Lee siempre el cuerpo: una interfaz SPA puede responder con HTML y `200` a una ruta inexistente.

Medir solo el estado

```
curl -sS -o /dev/null -w '%{http_code}\n' "$BASE/"
```

`-o /dev/null` descarta el cuerpo localmente, pero lo descarga. `-w` imprime una métrica después de la transferencia; `\n` añade un salto. Un `000` no es un estado HTTP del servidor: suele acompañar un fallo de conexión o transferencia. Comprueba también el error de curl.

Importante: curl normalmente no devuelve un código de salida de error solo por recibir HTTP 404. El estado HTTP y el estado del proceso son cosas distintas.

10 · curl: cabeceras, POST y datos

`-H` añade una cabecera a la petición. Para una API que acepta JSON, indica `Content-Type: application/json`. Para usar un bearer token, añade `Authorization: Bearer ....`

```
curl -i -X POST "$BASE/rest/user/login" \
  -H 'Content-Type: application/json' \
  -d '{"email":"alumno@example.test","password":"CLAVE_LAB"}'
```

Este es un **ejemplo de sintaxis**: la cuenta debe existir. `-X POST` selecciona el método. `-d` aporta el cuerpo y ya implica POST en este caso, por lo que `-X POST` es redundante pero sirve para explicitar la intención. Sin la cabecera JSON, curl usa normalmente un tipo de formulario para `-d` [C1].

Comillas y variables

Las comillas simples conservan el texto literal; las dobles permiten expandir variables. Por eso usamos comillas simples alrededor del JSON fijo y dobles alrededor de la cabecera con el token:

```
curl -sS "$BASE/api/Users/1" \
  -H "Authorization: Bearer $CUSTOMER_TOKEN"
```

Con comillas simples en esa cabecera enviarías literalmente `$CUSTOMER_TOKEN`. No añadas `-i` si vas a entregar el resultado a `jq`: las cabeceras no son JSON.

Evitar secretos en el historial

La contraseña escrita directamente en un comando puede quedar en el historial. Para la práctica de login utilizaremos lectura interactiva y `jq` para construir JSON correctamente. El token también es una credencial: quien lo posee puede usarlo mientras el servidor lo acepte.

Ejercicio: compara el cuerpo obtenido con `-i` y con `-sS`; identifica exactamente qué líneas impedirían que `jq` analizara la primera salida. Después añade `-o /dev/null -w` y observa que solo queda el código HTTP.

11 · Pipes, redirecciones y grep

Un pipe `|` conecta la salida estándar de un proceso con la entrada estándar del siguiente. No equivale a guardar un archivo ni incluye por defecto los errores, que viajan por stderr.

```
curl -sS "$BASE/" > index.html
grep -n 'script' index.html
printf '%s\n' 'login' 'logout' | grep 'login'
```

`>` crea o sobrescribe un archivo; `>>` añade contenido al final. `grep` selecciona líneas con coincidencias. `-n` añade el número de línea; `-i` ignora mayúsculas; `-o` imprime solo la parte coincidente; `-E` activa expresiones regulares extendidas. `sort -u` ordena y elimina duplicados.

Leer una cadena de herramientas

```
grep -oE '[A-Za-z0-9._/-]+\.js' index.html | sort -u
```

Primero se extraen coincidencias de un patrón de nombres JS; después se deduplican. Si no aparece nada, no concluyas que la web carece de JavaScript: quizá las rutas tienen otro formato, se generan dinámicamente o la petición no devolvió el HTML esperado.

Errores que el pipe puede ocultar

Por defecto, Bash devuelve como estado del pipe el del último comando. Para detectar fallos anteriores en scripts, usa `set -o pipefail` y comprueba el resultado. `grep` devuelve 1 cuando no encuentra coincidencias; eso no siempre es un fallo del laboratorio.

Comillas: escribe las expresiones regulares entre comillas simples para que Bash no interprete caracteres como `*` antes de que lleguen a grep.

Ejercicio: ejecuta cada tramo por separado, observa sus entradas y salidas y luego une el pipe. Esta técnica facilita localizar dónde se perdió un dato.

12 · Regex: clases y cuantificadores

El patrón [^"]+

```
printf '%s\n' 'src="main.js"' | grep -oE '[^"]+'
```

`[...]` define una clase. El `^` al principio, **dentro** de la clase, la niega: `[^"]` acepta cualquier carácter salvo comillas dobles. `+` exige una o más repeticiones. La salida contiene `src=` y `main.js`: el patrón no sabe cuál de los fragmentos es un atributo o una ruta.

Fuera de una clase, `^` suele anclar el inicio de línea. Dentro de la clase, las comillas no necesitan una barra inversa. Bash conserva el patrón porque está entre comillas simples.

El patrón [A-Za-z0-9._/-]+\.js

`A-Z`, `a-z` y `0-9` son rangos. Dentro de la clase, `.` es un punto literal, `_` un guion bajo y `/` una barra. El guion final es literal, no un rango. `+` exige al menos un carácter de ese conjunto. Fuera de la clase, `\.` exige un punto literal; `js` exige esas letras.

Ejemplo: de `src="/assets/main.abc123.js"` obtiene `/assets/main.abc123.js`. Es una heurística: puede recortar una URL absoluta, omitir parámetros `?v=2` o coincidir con el prefijo de `archivo.json`. No valida una URL ni analiza HTML completo.

Cuantificadores en grep -E

- `?`: cero o una repetición.
- `*`: cero o más; puede aceptar una secuencia vacía.
- `+`: una o más.
- `{3}`: exactamente tres.
- `{2,5}`: entre dos y cinco.
- `{2,}`: dos o más.

El cuantificador afecta al elemento inmediatamente anterior. `ab+` repite la `b`; `(ab)+` repite el grupo. El punto `.` fuera de una clase acepta cualquier carácter en la línea. Usamos `grep -E` para que `+`, paréntesis y llaves tengan esta interpretación sin escapes adicionales.

13 · Regex: contexto alrededor de loginForm

```
grep -oE '.{0,150}loginForm.{0,300}' app.js
```

`.{0,150}` admite entre cero y 150 caracteres antes del texto exacto `loginForm`. `.{0,300}` hace lo mismo para hasta 300 caracteres después. `-o` muestra solo cada fragmento coincidente. La coincidencia puede tener como máximo 459 caracteres: 150 + 9 + 300.

Es útil en JavaScript minificado, donde una línea puede contener miles de caracteres. Sin `-o`, grep podría imprimir la línea entera aunque el patrón esté acotado.

Límites y comportamiento

Los cuantificadores son codiciosos dentro de las reglas de coincidencia de grep: intentan abarcar tanto como permite el patrón. No garantizan una ventana independiente por cada aparición si varias están cerca; las coincidencias que imprime grep no se solapan.

GNU grep trabaja normalmente línea a línea. El punto no atraviesa el salto de línea en este uso. Si `loginForm` está al principio o al final, puede haber menos contexto. Si está renombrado por minificación, no aparecerá aunque exista un formulario.

```
grep -n 'loginForm' app.js
grep -oE '.{0,150}/rest/user/login.{0,300}' app.js
```

La segunda búsqueda usa una ruta concreta como pista alternativa. Un fragmento alrededor de una palabra sirve para orientarse, pero no reemplaza leer el flujo del programa o observar la petición real en el navegador.

Ejercicio de interpretación

¿Por qué `.{150}` sería peor para esta tarea? Porque exige exactamente 150 caracteres y perdería una coincidencia demasiado cercana al comienzo de la línea. ¿Por qué `.*loginForm.*` puede ser incómodo? Porque en un bundle minificado puede imprimir prácticamente todo el archivo.

14 · Enumeración de recursos JavaScript

Contexto: herramientas. Descarga el HTML y observa los recursos que declara:

```
curl -sS "$BASE/" -o index.html
grep -oE 'src="[^"]+"' index.html
grep -oE '[A-Za-z0-9._/-]+\.js' index.html | sort -u
```

El primer patrón une `src="`, una secuencia sin comillas y la comilla final. Puede incluir imágenes u otros recursos. El segundo intenta aislar rutas con `.js`. Estos patrones presuponen un formato sencillo: no cubren comillas simples, todos los atributos ni todos los formatos válidos de HTML.

Descargar un recurso seleccionado

La salida podría contener `main.abc123.js`; es un nombre ilustrativo. Copia una ruta **real de tu salida**, verifica que pertenece al mismo laboratorio y descárgala:

```
JS_PATH='/main.abc123.js'  # sustituir por la ruta observada
curl -sS "$BASE$JS_PATH" -o app.js
head -c 200 app.js
```

Si obtienes HTML, quizá la ruta no existe y la SPA devuelve su página inicial. Consulta cabeceras y estado antes de analizar el archivo como JavaScript. No descargues automáticamente todos los dominios que aparezcan: pueden ser servicios externos fuera de alcance.

Qué aporta el código del cliente

Un bundle puede revelar rutas, nombres de campos, validaciones de interfaz y cómo se construyen peticiones. Su contenido es accesible al navegador y no debe contener secretos de servidor.

Una ruta encontrada es una **pista**, no prueba de que esté activa ni de que sea vulnerable. Puede ser código antiguo, un comentario, una ruta parcial o una URL construida por concatenación. Los módulos cargados después de interactuar con la página pueden no aparecer en el HTML inicial.

Ejercicio: anota nombre del archivo, ruta, fecha y palabra que permitió localizar el login. Separa la pista estática de la petición que verificarás después.

15 · Endpoints /api y JSON con jq

Busca rutas candidatas en el archivo descargado:

```
grep -oE '/api/[A-Za-z0-9_/?=&.-]+' app.js | sort -u
grep -oE '/rest/[A-Za-z0-9_/?=&.-]+' app.js | sort -u
```

El patrón exige el prefijo y al menos un carácter admitido. Puede cortar rutas dinámicas, omitir segmentos o capturar texto adicional. `/api/Users` y `/rest/user/login` son rutas de interés del curso, pero no se afirma que ambos comandos las encuentren en cualquier versión.

jq entiende la estructura JSON

```
printf '%s\n' '{"data":{"id":1,"role":"admin"}}' | jq .
printf '%s\n' '{"data":{"id":1,"role":"admin"}}' \
  | jq '.data | {id,role}'
```

`.` muestra el documento con formato. `.data` entra en esa propiedad. `{id,role}` construye un objeto con solo esas claves; no cambia la respuesta del servidor. El pipe **dentro de las comillas** pertenece a jq; el pipe entre procesos pertenece a Bash [J1].

Filtros del curso

- `.data[]`: emite cada elemento de un array `data`.
- `.data | {id,email,role}`: proyecta un usuario individual.
- `.data[] | {id,email,role}`: proyecta cada usuario de una lista.
- `.authentication.token`: accede al token del login si esa estructura existe.
- `-r`: devuelve cadenas sin comillas JSON.
- `-e`: permite usar el resultado para detectar `false`, `null` o ausencia de resultados mediante el estado de salida.

Si aparece `parse error`, comprueba si recibiste HTML o cabeceras. Si obtienes `null`, revisa el nombre y la ubicación del campo. Si `.data[]` falla, verifica primero si `data` es un array, un objeto o si la respuesta es un error.

16 · Login y creación de TOKEN

En el navegador, registra una cuenta de laboratorio mediante el formulario de Juice Shop. Este procedimiento no cambia su rol. Usa las credenciales de una cuenta que controles; para la prueba final necesitarás una cuenta normal `customer`.

Contexto: herramientas, Bash. Construye JSON sin concatenar a mano los caracteres de la contraseña:

```
read -r -p 'Email de laboratorio: ' LAB_EMAIL
read -r -s -p 'Contraseña: ' LAB_PASSWORD
printf '\n'
LOGIN_RESPONSE=$(
  printf '%s' "$LAB_PASSWORD" \
  | jq -Rs --arg email "$LAB_EMAIL" \
      '{email:$email,password:.}' \
  | curl -sS -X POST "$BASE/rest/user/login" \
      -H 'Content-Type: application/json' -d @-
)
unset LAB_PASSWORD
TOKEN=$(printf '%s' "$LOGIN_RESPONSE" \
  | jq -er '.authentication.token | strings | select(length > 0)')
```

`read -s` oculta lo tecleado. `jq -Rs` lee la contraseña como texto y `--arg` aporta el correo como cadena; jq escapa comillas y otros caracteres al generar JSON. `-d @-` lee ese JSON de la entrada estándar. `$(...)` captura la salida en una variable.

Comprueba si la asignación del token tuvo éxito **inmediatamente**:

```
if [ $? -eq 0 ] && [ -n "$TOKEN" ]; then
  printf 'Token recibido\n'
else
  printf 'Login fallido: revisar estado HTTP y estructura JSON\n'
fi
unset LOGIN_RESPONSE
```

Una respuesta exitosa puede contener `authentication.token` y otros campos; depende de la versión. No imprimas el token para demostrar que existe. Si falla el análisis, inspecciona el estado HTTP y el mensaje de error sin publicar credenciales. El nombre `TOKEN` no indica que el usuario sea administrador: solo almacena el resultado de ese login.

17 · JWT, Base64URL y RS256

Un JWT firmado en formato compacto JWS suele tener tres segmentos separados por puntos:

```
header.payload.signature
```

El **header** declara parámetros como `alg` y, a menudo, `typ`. El **payload** contiene claims, por ejemplo un identificador, rol o expiración. Su estructura exacta depende de la aplicación. La **signature** permite verificar integridad y procedencia con la clave adecuada [T1].

Header y payload se representan con Base64URL: usa `-` y `_` en lugar de `+` y `/` y normalmente omite el relleno `=`. **Codificar no es cifrar.** Cualquiera que tenga el token puede leer esos segmentos. No todos los JWT usan este formato de tres partes; los JWT cifrados tienen otra estructura.

Inspección local sin enviar el token a terceros

```
printf '%s' "$TOKEN" | python3 -c '
import sys, json, base64
parts = sys.stdin.read().strip().split(".")
if len(parts) != 3:
    raise SystemExit("Se esperaba un JWS de tres segmentos")
for label, value in zip(("header", "payload"), parts[:2]):
    raw = base64.urlsafe_b64decode(value + "=" * (-len(value) % 4))
    print(label, json.dumps(json.loads(raw), indent=2))
'
```

El relleno se restaura hasta un múltiplo de cuatro. La salida puede contener datos sensibles: mantenla local. **Este comando decodifica; no verifica la firma.** Un payload legible no demuestra que el servidor lo acepte.

Si el header observado indica `RS256`, significa firma RSA con RSASSA-PKCS1-v1_5 y SHA-256 [T2]. El emisor firma con una clave privada y el verificador utiliza la pública. No significa que el payload esté cifrado. Cambiar `role` en el texto rompe la firma salvo que exista un fallo adicional de validación.

El historial recuperado no incluye el header real: RS256 se explica como concepto solicitado, no como algoritmo confirmado de tu token. El servidor debe fijar algoritmos aceptados y validar firma, vigencia y claims relevantes para su aplicación.

18 · Tokens y almacenamiento del navegador

Dónde puede guardar una aplicación la sesión

localStorage almacena pares clave/valor por origen y normalmente persiste entre sesiones del navegador. JavaScript del mismo origen puede leerlo. **sessionStorage** se asocia al origen y a la sesión de una pestaña; suele desaparecer al cerrar esa sesión. Tampoco está protegido frente a scripts del mismo origen.

Cookies se envían automáticamente en peticiones que cumplen sus reglas. `HttpOnly` impide su lectura desde JavaScript; `Secure` restringe su envío a contextos seguros según las reglas del navegador; `SameSite` limita ciertos envíos entre sitios. Las cookies de autenticación requieren considerar CSRF. `HttpOnly` reduce el robo directo de la cookie por scripts, pero no elimina todos los efectos de XSS.

Ni localStorage ni sessionStorage envían automáticamente una cabecera bearer: normalmente lo hace el código de la aplicación. Evita asumir dónde se guarda el JWT sin inspeccionarlo.

Observar el login en el navegador

Abre las herramientas de desarrollo en tu Juice Shop local. En Network, realiza un login con la cuenta de prácticas y localiza `/rest/user/login`. Revisa método, campos enviados, estado y estructura de respuesta. En Application o Storage, inspecciona el almacenamiento del mismo origen. Los nombres de pestañas y claves pueden variar.

No pegues tokens en decodificadores públicos ni compartas capturas sin ocultarlos. El historial recuperado no confirma un nombre concreto de clave ni una ubicación de almacenamiento.

Variables de Bash no son almacenamiento web

`TOKEN` y `CUSTOMER_TOKEN` existen en la shell donde se asignan. No sobreviven automáticamente a otra sesión ni cambian el login del navegador. No necesitan `export` para que Bash expanda su valor en un argumento de curl. Exportarlas amplía su disponibilidad a procesos hijos.

Al terminar: `unset TOKEN CUSTOMER_TOKEN`. Cerrar una sesión de navegador no garantiza que un bearer token ya emitido haya quedado revocado en el servidor.

19 · Identidad y permisos: preparar la comparación

Autenticación: determinar qué identidad presenta una petición. **Autorización:** decidir si esa identidad puede realizar esta acción sobre este recurso. Un token válido no concede acceso universal.

Para crear `CUSTOMER_TOKEN`, repite el login del capítulo 16 con una cuenta normal cuya identidad conozcas. Después:

```
CUSTOMER_TOKEN="$TOKEN"
unset TOKEN
```

Comprueba que esa cuenta es `customer` mediante el contexto de registro y la información de cuenta que expone el laboratorio. Decodificar sus claims puede ayudar a inspeccionar su identidad, pero no es una validación criptográfica independiente. No reutilices por accidente un token de otra cuenta privilegiada.

Matriz de pruebas propuesta

- Sin token: comprobar si la API exige autenticación.
- Customer A sobre su propio ID: comprobar el acceso permitido.
- Customer A sobre Customer B: comprobar propiedad del objeto.
- Customer A sobre ID 1: comprobar acceso a otro usuario, que en la evidencia es administrador.
- Administrador: solo como comparación si dispones de una cuenta autorizada de ese tipo.

Registra para cada prueba la identidad, método, ruta, estado HTTP y campos visibles. La política esperada del ejercicio es que un cliente no lea perfiles ajenos ni liste indiscriminadamente usuarios.

Consultar no es convertirse en administrador

Leer un objeto con `role: admin` no convierte al solicitante en administrador. La evidencia muestra divulgación de datos; no demuestra cambios de rol, escritura, acceso a contraseñas ni toma de cuentas.

Acceso horizontal describe el salto entre objetos de usuarios equivalentes. Acceso vertical describe acciones reservadas a un nivel superior. El acceso por ID se analiza como autorización a nivel de objeto; el listado completo puede implicar además controles a nivel de función. No los confundas únicamente por el rol del registro devuelto.

20 · Demostración de Broken Access Control / IDOR

Contexto: herramientas, con el token de cliente de la sesión:

```
curl -sS -o /dev/null -w '%{http_code}\n' \
  -H "Authorization: Bearer $CUSTOMER_TOKEN" \
  "$BASE/api/Users"

curl -i \
  -H "Authorization: Bearer $CUSTOMER_TOKEN" \
  "$BASE/api/Users/1"
```

Evidencia observada: el listado devolvió `200`. La consulta individual devolvió `HTTP/1.1 200 OK` y datos del usuario 1. Extracto reducido de la respuesta real:

```
{
  "status": "success",
  "data": {
    "id": 1,
    "username": "",
    "email": "admin@juice-sh.op",
    "role": "admin",
    "isActive": true
  }
}
```

La respuesta original también contenía campos de perfil, fechas y otros metadatos. Su cabecera `Date` era `Sat, 07 Nov 2026 17:13:11 GMT` (18:13:11 en Zúrich). No contenía un campo de contraseña en la evidencia mostrada.

Qué demuestra y qué falta verificar

Bajo la identidad customer indicada en la sesión y la política de perfiles privados del ejercicio, leer ese registro ajeno evidencia control de acceso roto. Manipular el identificador de la ruta lo encuadra como IDOR; en APIs se relaciona con BOLA, autorización rota a nivel de objeto [O1].

Un `200` aislado en `/api/Users` no bastaba: era necesario revisar el contenido. Tampoco `Access-Control-Allow-Origin: *` explica por sí solo este resultado: curl no aplica la política CORS del navegador y CORS no sustituye los permisos del servidor.

Para una auditoría formal, añade la prueba de identidad de la cuenta solicitante, su ID, la política esperada y una captura reproducible con el bearer token oculto.

21 · Enumeración limitada y conservación de evidencia

La prueba de la conversación recorría cinco identificadores:

```
for id in 1 2 3 4 5; do
  echo "Usuario $id:"
  curl -sS \
    -H "Authorization: Bearer $CUSTOMER_TOKEN" \
    "$BASE/api/Users/$id" \
  | jq '.data | {id,email,role}'
done
```

`for` asigna a `id` cada valor; `do` comienza el bloque y `done` lo termina. Las comillas dobles permiten expandir `$id` y el token. `jq` muestra solo los campos relevantes.

Resultado comunicado: el alumno confirmó «ha devuelto los 5 usuarios». No se conserva la salida íntegra de esos cinco registros, por lo que no se inventan sus correos ni roles. La respuesta completa visible corresponde al usuario 1.

Guardar una reproducción mínima

```
umask 077
mkdir -p evidencia
curl -sS -D evidencia/users-1.headers \
  -o evidencia/users-1.json \
  -H "Authorization: Bearer $CUSTOMER_TOKEN" \
  "$BASE/api/Users/1"
jq '.data | {id,email,role}' evidencia/users-1.json \
  > evidencia/users-1-resumen.json
```

`-D` guarda las cabeceras **de respuesta**, no la cabecera Authorization enviada. `umask 077` restringe por defecto el acceso a archivos nuevos. Revisa también las cabeceras de respuesta antes de compartirlas: podrían contener cookies u otros datos sensibles.

Conserva el original de manera restringida y adjunta al informe solo el extracto necesario. Registra fecha, versión o digest de la imagen, cuenta solicitante sin secretos, petición redactada y comportamiento esperado. No edites el original para que coincida con una hipótesis.

Límite: esta prueba muestra lectura de registros existentes. No demuestra que todos los IDs posibles funcionen ni autoriza a probar modificaciones o borrados.

22 · Informe del hallazgo

LAB-001 · Un cliente puede leer perfiles ajenos por ID

Activo y alcance: API de Juice Shop, laboratorio Docker propio. `GET /api/Users/:id`. Fecha de evidencia: 7 de noviembre de 2026.

Descripción: usando la sesión identificada como customer, la petición al usuario 1 devolvió datos de `admin@juice-sh.op`. El alumno comunicó que la iteración de IDs 1 a 5 devolvió los cinco usuarios.

Precondiciones: cuenta normal autenticada, token vigente y conectividad con el servicio del laboratorio. Para formalizar la evidencia, registrar el ID y rol verificados de la cuenta solicitante, sin adjuntar su token.

Pasos de reproducción

1. Iniciar sesión con una cuenta customer propia y guardar el token.
2. Solicitar `GET /api/Users/1` con bearer token.
3. Comprobar `200 OK` y los campos `id`, `email` y `role` del registro ajeno.
4. Repetir de forma limitada sobre los cinco IDs del ejercicio.

Resultado esperado: denegar la lectura de perfiles ajenos, salvo permiso explícito conforme a la política del sistema. Puede utilizarse `403` o un `404` uniforme para no revelar existencia.

Resultado observado: respuesta de éxito con datos del usuario 1. Confirmación verbal de cinco resultados en la enumeración. El `200` del listado se registra como observación complementaria; su cuerpo completo no está conservado.

Impacto demostrado: divulgación de correos, roles y metadatos de perfiles ajenos. La enumeración facilita conocer cuentas y perfiles privilegiados. No se ha demostrado acceso de escritura ni compromiso de cuentas.

Severidad provisional: media para la divulgación observada; reevaluar según sensibilidad, volumen, exposición y requisitos de autenticación. No asignar una puntuación CVSS cerrada sin justificar todos sus componentes.

Clasificación y evidencia: Broken Access Control; IDOR/BOLA para la ruta individual. Adjuntar petición redactada, respuesta mínima y matriz de identidades. El estado de mitigación queda pendiente: aquí no se ha modificado el servidor.

23 · Mitigación y prueba de regresión

El servidor debe autorizar cada operación sobre cada objeto, aunque la petición lleve un token válido. La identidad y los permisos deben proceder de una sesión validada, no de campos que el cliente pueda elegir [O1].

Política orientativa para una aplicación propia

```
actor = verificar_sesion(peticion)
si no hay actor: responder 401
objetivo = buscar_usuario(id_solicitado)
si no existe objetivo: responder 404
si actor.id != objetivo.id y no tiene_permiso(actor, "leer_usuarios"):
    responder 403
responder solo_campos_permitidos(actor, objetivo)
```

Es pseudocódigo pedagógico, no un parche de Juice Shop. Una API multiempresa necesitaría además comprobar organización o tenant. La autorización puede incorporarse a la consulta para impedir que se recuperen objetos fuera de alcance.

Medidas complementarias

Restringe el listado `/api/Users` con permisos explícitos y filtra filas y campos. Aplica denegación por defecto y centraliza políticas para que no dependan de recordar controles en cada ruta. Añade registros de intentos denegados sin almacenar tokens.

Los identificadores aleatorios reducen la facilidad de adivinar objetos, pero no reemplazan autorización. Ocultar botones, cambiar el frontend o limitar CORS tampoco impide peticiones directas. El rate limiting reduce abuso; no corrige un acceso indebido permitido.

Cómo verificar la corrección

Repite la matriz: sin token, token inválido o vencido, usuario propietario, usuario ajeno y rol con permiso. Comprueba estado **y cuerpo**. Un rechazo no debe seguir filtrando datos. Mantén pruebas automatizadas donde Customer A no pueda leer a Customer B y donde el acceso legítimo siga funcionando.

En el laboratorio deliberadamente vulnerable, conserva la instancia de estudio. Puedes practicar la corrección en una aplicación de ejemplo separada y comparar resultados sin perder la evidencia original.

24 · Repaso, diagnóstico y próximos módulos

Repaso con respuestas breves

¿HEAD y GET sin mostrar cuerpo son lo mismo? No: `-I` cambia el método; `-o /dev/null` descarta localmente el cuerpo descargado.

¿Decodificar JWT prueba su autenticidad? No. Para eso hay que validar la firma y los claims aplicables.

¿Leer un perfil admin concede su rol? No. Demuestra lectura de ese recurso, no adquisición de privilegios.

¿Un `200` prueba un IDOR? No por sí solo. Hay que vincular identidad, política, objeto ajeno y datos devueltos.

Diagnóstico rápido

`Could not resolve host`: revisa el contexto y la red Docker. `Connection refused`: revisa puerto, servicio y arranque. HTML en lugar de JSON: revisa ruta y respuesta SPA. `jq: parse error`: separa cabeceras y cuerpo. Token vacío o `null`: revisa el login y la estructura de respuesta. `401`: vuelve a comprobar identidad y vigencia. Un archivo JS distinto: enumera los recursos actuales en vez de reutilizar un nombre antiguo.

Próximos módulos

1. Consolidar la matriz de permisos con dos cuentas customer y evidencias completas.
2. Comparar cookies, bearer tokens, expiración y cierre de sesión.
3. Observar peticiones con un proxy de laboratorio y reproducirlas de forma controlada.
4. Estudiar validación de entradas e inyecciones exclusivamente en retos locales.
5. Analizar XSS y diferencias entre almacenamiento accesible a scripts y cookies HttpOnly.
6. Construir una API de ejemplo con autorización correcta y pruebas de regresión.
7. Mejorar reproducibilidad: imágenes fijadas, reinicio de datos y registro de versiones.

Estos módulos son propuestas futuras, no prácticas ya realizadas.

Plantilla para ampliar el curso

Añade a cada módulo: objetivo; requisitos; contexto de ejecución; comandos; explicación; resultado esperado; evidencia observada; errores frecuentes; mitigación; preguntas de repaso. Mantén un registro de cambios con fecha y versión. La edición 1.0 documenta el temario solicitado y las pruebas finales disponibles.

25 · Fuentes y referencias

Las referencias técnicas se consultaron el 7 de noviembre de 2026. Las versiones de las herramientas pueden cambiar. Los ejemplos son material pedagógico propio; las respuestas reales están identificadas como tales.

Fuente de la práctica: conversación «Curso de hacking ético Docker», identificador `6aee3777-8950-83ed-8878-0dce8633536d`. La recuperación disponible contiene los intercambios finales, sin adjuntos ni acceso a los primeros módulos. No se incorporaron credenciales ni tokens.

[D1] Docker Engine en Ubuntu. Requisitos, instalación y consideraciones del firewall.

https://docs.docker.com/engine/install/ubuntu/

[D2] Publicación de puertos de Docker. Interfaces, localhost y limitaciones por versión.

https://docs.docker.com/engine/network/port-publishing/

[D3] Redes de Docker Compose. Servicios, redes y atributo internal.

https://docs.docker.com/reference/compose-file/networks/

[N1] Nmap: detección de servicios y versiones.

https://nmap.org/book/man-version-detection.html

[C1] Manual de curl. Opciones de peticiones y formato de salida.

https://curl.se/docs/manpage.html

[J1] Manual de jq. Filtros, construcción de objetos y opciones de ejecución.

https://jqlang.org/manual/

[T1] RFC 7519: JSON Web Token.

https://www.rfc-editor.org/rfc/rfc7519

[T2] RFC 7518: JSON Web Algorithms. Definición de RS256.

https://www.rfc-editor.org/rfc/rfc7518

[O1] OWASP API Security: API1:2023 BOLA. Autorización por objeto.

https://api-security.owasp.org/editions/2023/en/0xa1-broken-object-level-authorization/

Lectura adicional sobre grep: manual de GNU grep, para profundizar en expresiones regulares básicas y extendidas.

https://www.gnu.org/software/grep/manual/grep.html